



**VIT<sup>®</sup>**  
**BHOPAL**  
[www.vitbhopal.ac.in](http://www.vitbhopal.ac.in)

# **PROJECT REPORT**

## **MOVIE RECOMMENDER APP**

**Name: Abhigyan Tiwari**

**Reg no: 25BAI11270**

**Branch: CSE ( AI-ML )**

**Course: PROGRAMMING IN JAVA(CSE2006)**

# **INTRODUCTION**

In the modern digital era, users are exposed to a vast amount of entertainment content, making it difficult to choose relevant movies. A recommendation system helps users by suggesting movies based on their preferences.

This project implements a **Movie Recommendation System using Java and content-based filtering**, where recommendations are generated based on similarities between movies using features such as **genres, keywords, and overview**.

The system uses a movie dataset stored in movies.csv. Java is used to process the dataset, extract relevant features, calculate similarity between movies, and provide recommendations through an interactive **Java Swing GUI**.

The system is also integrated with the **OMDb API** to retrieve additional movie information such as posters, release year, director, actors, and IMDb rating.

---

## **MOTIVATION OF THE PROJECT**

- To simplify movie selection for users
  - To understand practical implementation of machine learning
  - To explore real-world applications of AI
  - To build an interactive GUI-based application
-

## **PROBLEM STATEMENT**

Users often struggle to find movies that match their interests due to the large number of available options. Traditional browsing is inefficient and time-consuming.

This project aims to develop a system that:

- Takes a movie as input
- Suggests similar movies automatically

---

## **OBJECTIVE OF THE PROJECT**

- To build a movie recommendation system using Java
- To implement content-based filtering
- To calculate movie similarity using cosine similarity
- To create a user-friendly GUI using Java Swing
- To process movie information from a CSV dataset
- To fetch real-time movie details using the OMDb API
- To display movie posters and relevant information to the user

---

## **EXISTING METHODS**

1. Content-Based Filtering
2. Collaborative Filtering
3. Hybrid Systems

---

## **PROS AND CONS OF THE STATED METHODS**

### **Content-Based Filtering**

#### **Pros:**

- No need for user data
- Works for new users

#### **Cons:**

- Limited diversity
- Depends on dataset quality

---

### **Collaborative Filtering**

#### **Pros:**

- More personalized recommendations

#### **Cons:**

- Cold start problem
- Requires user history

---

### **Hybrid Systems**

#### **Pros:**

- More accurate

#### **Cons:**

- Complex implementation

---

# **HARDWARE AND SOFTWARE REQUIREMENTS**

## **Hardware:**

- Minimum 4GB RAM
- Basic computer/laptop

## **Software:**

- **Java JDK 11 or above**
- **Visual Studio Code / IntelliJ IDEA / Eclipse**
- Java Swing
- Java Collections Framework
- Java HTTP Client
- Java ImageIO
- Dataset: movies.csv
- Internet connection for OMDb API

# **METHODOLOGY AND GOAL**

## **Methodology:**

The system follows these major steps:

1. Load the movie dataset
2. Read and process movie information
3. Handle missing values
4. Extract relevant features
5. Combine genres, keywords, and overview
6. Convert textual information into numerical representations
7. Calculate similarity between movies
8. Select the top 5 similar movies
9. Fetch additional movie information using OMDb API
10. Display recommendations through Java Swing GUI

## **Goal:**

The main goal of the project is to provide **fast and relevant movie recommendations through a simple Java-based graphical interface.**

# **FUNCTIONAL MODULES DESIGN AND ANALYSIS**

## **1. Data Processing Module**

- Handles dataset loading and cleaning

## **2. Feature Extraction Module**

- Converts text to numerical format

## **3. Similarity Module**

- Computes cosine similarity

## **4. Recommendation Module**

- Suggests top 5 similar movies

## **5. API Module**

- Fetches movie details and posters

## **6. GUI Module**

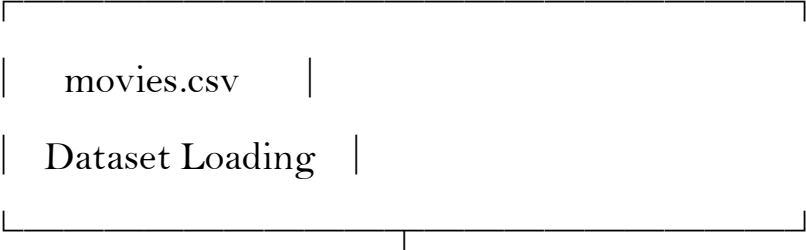
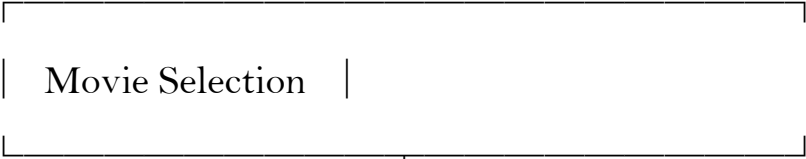
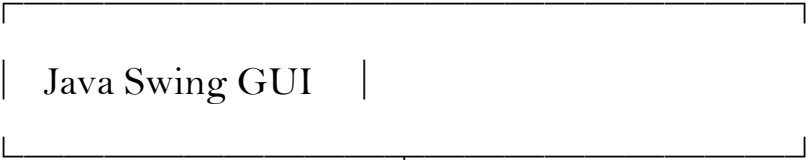
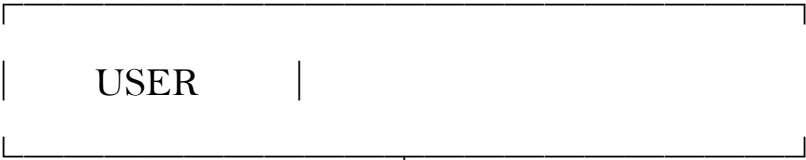
- The GUI module is developed using **Java Swing**.

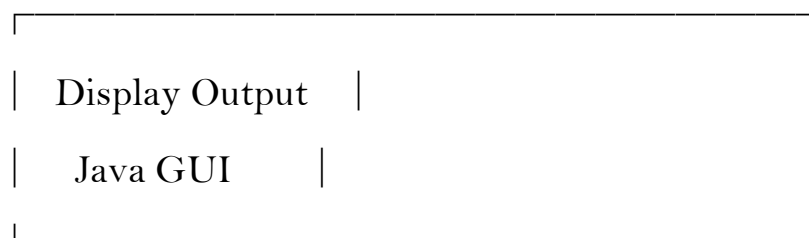
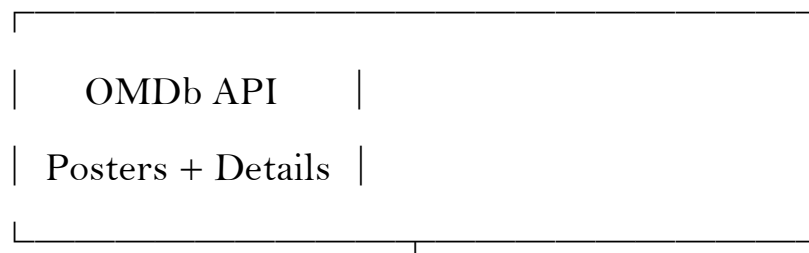
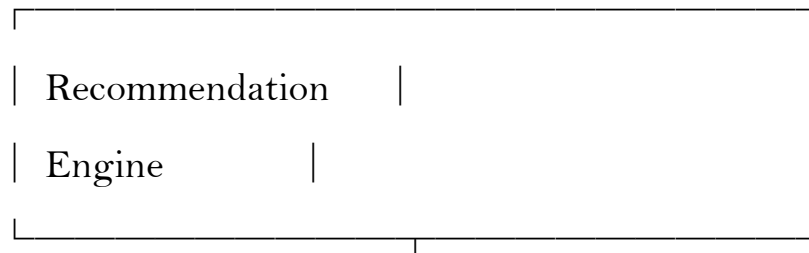
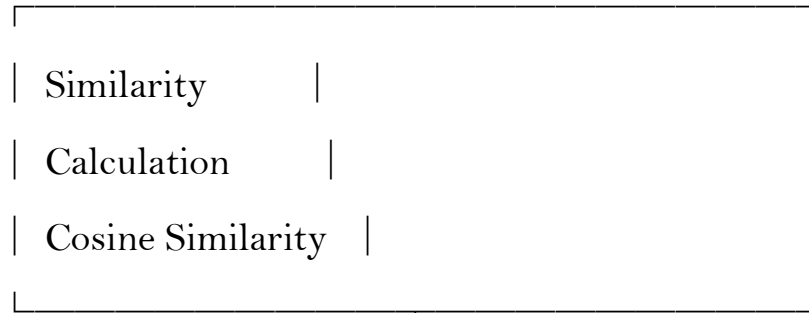
# **ALGORITHM DEVELOPMENT**

## **Step-by-step Algorithm:**

1. Start the application.
  2. Load the movies.csv dataset.
  3. Identify the required movie columns.
  4. Handle missing movie information.
  5. Extract genres, keywords, and overview.
  6. Combine the selected features into a single representation.
  7. Convert the textual information into numerical feature vectors.
  8. Select a movie from the Java Swing interface.
  9. Find the selected movie in the dataset.
  10. Calculate similarity between the selected movie and other movies.
  11. Sort the movies according to similarity scores.
  12. Select the top 5 similar movies.
  13. Send movie titles to the OMDb API.
  14. Retrieve movie details and poster information.
  15. Display the recommendations and details in the GUI.
  16. End.
-

# SOFTWARE ARCHITECTURAL DIAGRAM





## **CODING**

The system is implemented using **Java**.

### **Technologies used in implementation:**

- **Java** – Core programming language
- **Java Swing** – Graphical User Interface
- **Java Collections Framework** – Data storage and processing
- **Java Regular Expressions** – Parsing and extracting required information
- **Java HTTP Client** – API communication
- **Java ImageIO** – Processing movie poster images
- **OMDb API** – Retrieving movie details
- **CSV Dataset** – Storing movie information

The complete implementation is contained in the Main.java file.

The Java program performs dataset loading, feature extraction, cosine similarity calculation, recommendation generation, GUI interaction, API communication, and poster display.

---

## **OUTPUT**

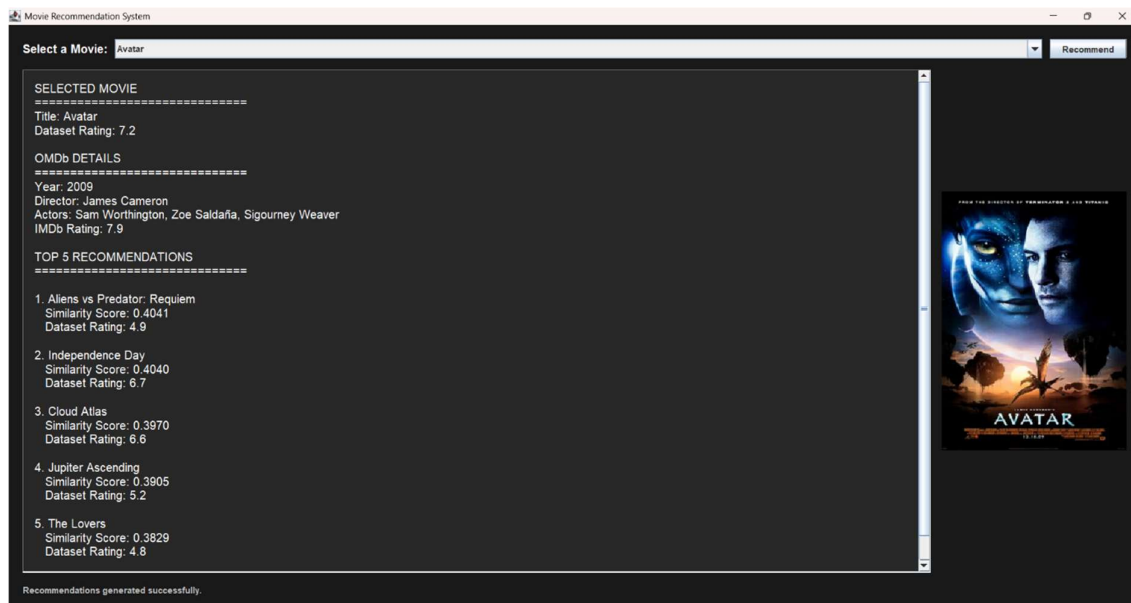
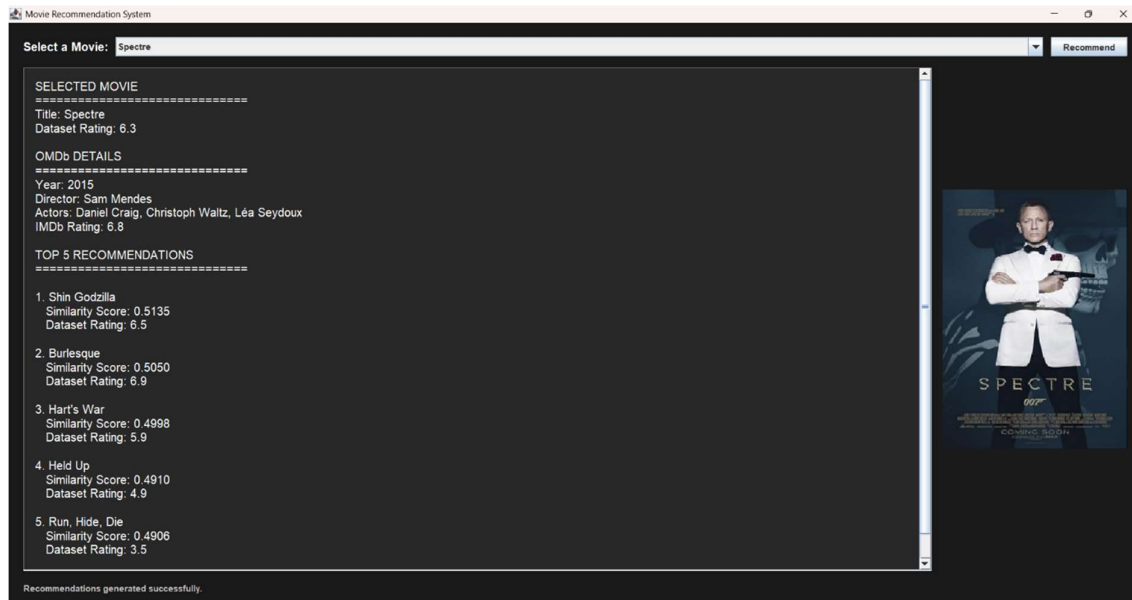
- User selects a movie
- System displays:
  - Top 5 recommended movies
  - Movie poster
  - Year, Director, Actors
  - IMDb rating

---

## **KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM**

- Content-based movie recommendation
  - Cosine similarity calculation
  - CSV dataset processing using Java
  - Interactive GUI using Java Swing
  - OMDb API integration
  - Movie poster retrieval
  - Movie information display
  - Top 5 movie recommendation generation
-

# SCREENSHOTS



## **SIGNIFICANT PROJECT OUTCOMES**

- Successfully developed a working movie recommendation system using Java.
  - Implemented content-based recommendation techniques.  
Implemented cosine similarity for finding similar movies.
  - Created an interactive Java Swing GUI.
  - Successfully processed a movie CSV dataset.
  - Integrated the OMDb API for additional movie information.
  - Displayed movie posters and details through the application.
- 

## **TESTING AND REFINEMENT**

- Tested with multiple movie inputs
  - Handled missing data
  - Improved GUI responsiveness
  - Fixed API errors and invalid inputs
-

## **PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS**

- Netflix, Amazon Prime recommendations
- E-commerce product suggestions
- Music recommendation systems
- Online content platforms

---

## **CONTRIBUTION / FINDINGS OF THE PROJECT**

- How content-based recommendation systems work
- How movie features can be used for similarity calculation
- The importance of feature extraction
- Practical implementation of cosine similarity
- Integration of an external API with Java
- Development of a complete GUI-based application
- Practical use of Java programming and object-oriented concepts

---

## **LIMITATION / CONSTRAINTS OF THE SYSTEM**

- No user personalization
- Depends on dataset quality
- Limited recommendation diversity
- Requires internet for API

---

## **CONCLUSION**

The project successfully demonstrates the development of a **Movie Recommendation System using Java** and content-based filtering techniques.

The system analyzes movie information such as genres, keywords, and overview to determine similarities between movies. Cosine similarity is used to identify movies that have similar content characteristics.

A **Java Swing graphical user interface** provides a simple way for users to select a movie and view recommendations. The integration of the **OMDb API** further enhances the application by providing movie posters and additional information such as release year, director, actors, and IMDb rating.

Overall, the project demonstrates how Java programming, recommendation algorithms, data processing, GUI development, and API integration can be combined to create a practical movie recommendation application.

---

## **FUTURE ENHANCEMENTS**

- Add collaborative filtering
  - Improve UI design
  - Deploy as web app (Streamlit/Flask)
  - Add user login system
  - Use deep learning models
-

## **REFERENCES**

1. Oracle Java Documentation
2. Java Swing Documentation
3. Java HTTP Client Documentation
4. OMDb API Documentation
5. Java ImageIO Documentation
6. Kaggle – Movie Dataset Resources
7. GitHub – Java Project Development Resources